CÓDIGOCLARO · RUTA JAVA

Tema 08 · Clases y objetos

Guía de estudio · DAM/DAW

Estado, comportamiento, instancias y encapsulación.

1. Explicación

Una clase define estado y comportamiento; un objeto es una instancia con sus propios campos. Encapsular limita cómo se cambia ese estado y protege invariantes. Los campos static pertenecen a la clase; los de instancia pertenecen a cada objeto.

2. Ejemplo

Código · java

```
class Contador {
    private int valor;
    void incrementar() { valor++; }
    int obtenerValor() { return valor; }
}
public class Main {
    public static void main(String[] args) {
        Contador a = new Contador(), b = new Contador();
        a.incrementar();
        System.out.println(a.obtenerValor() + ", " + b.obtenerValor());
    }
}
```

Resultado esperado

```
1, 0
```

3. Errores frecuentes

- Hacer públicos todos los campos y permitir estados inválidos.
- Confundir la definición de clase con la instancia creada por new.
- Usar static para estado que debe variar por objeto.

4. Ejercicio

Modela una lámpara con estado apagado/encendido y una operación para alternarlo.

Casos que debe superar

Entrada	Resultado esperado
recién creada	apagada
alternar()	encendida
alternar() de nuevo	apagada

5. Solución explicada

Código · java

```
class Lampara {
    private boolean encendida;
    void alternar() { encendida = !encendida; }
    String estado() { return encendida ? "encendida" : "apagada"; }
}
public class Main {
    public static void main(String[] args) { Lampara l = new Lampara(); l.alternar(); System.out.
println(l.estado()); }
}
```

Otras soluciones válidas

- Usar enum para más de dos estados.

• Preferir acciones encender/apagar si alternar puede ser ambiguo.

6. Fuentes oficiales

Esta guía resume conceptos para estudiar. Consulta las fuentes para comprobar detalles de versión y APIs.

Dev.java — Classes and Objects
Dev.java — Objects, Classes, Interfaces, Packages, and Inheritance